```
> # ============================================================
  # Maple Transcript: Photon Cylindrical Wave — TauTimp Framework
  # Variable-Coefficient Wave Equation in Cylindrical Coordinates
  # Author: Viorel-Mihai Popescu
  # Date: May 2026
  # ============================================================

  restart;
  with(plots):
  with(plottools):
  with(VectorCalculus):

  # ============================================================
  # SECTION 1: PHYSICAL CONSTANTS AND PARAMETERS
  # ============================================================

  printf("=== TauTimp Photon Cylindrical Wave ===\n");
  printf("Loading physical constants...\n\n");

  # Fundamental constants
  c_light := 2.998e8:          # speed of light [m/s]
  R_TT    := 4.235518529e22:       # TauTimp characteristic radius
  [m]
  hbar    := 1.0546e-34:       # reduced Planck [J.s]
  lambda0 := 5.0e-7:           # reference wavelength [m] (visible
  photon)

  # Derived photon parameters
  omega0  := 2*Pi*c_light/lambda0:
  k0      := 2*Pi/lambda0:
  mu_ref  := k0:               # reference radial wavenumber

  printf("c         = %.4e m/s\n", c_light);
  printf("R_TauTimp = %.4e m\n",   R_TT);
  printf("lambda0   = %.4e m\n",   lambda0);
  printf("omega0    = %.4e rad/s\n", evalf(omega0));
  printf("k0        = %.4e 1/m\n",  evalf(k0));
  printf("\n");

  # ============================================================
  # SECTION 2: RADIAL EIGENFUNCTION H3(r)
  # ============================================================
```

```
printf("=== SECTION 2: Radial Eigenfunction H3(r) ===\n");

# The radial equation after separation:
#   d^2H3/dr^2 + (1/r)*dH3/dr + (mu^2*r/R)*H3 = 0
#
# Liouville transformation: eta = (2*mu/3)*sqrt(r^3/R)
#   H3(r) = r^(-1/2) * [C1*BesselJ(1/3,eta) + C2*BesselY(1/3,eta)
]

# Argument of Bessel function
eta_func := proc(r_val, mu_val, R_val)
    return (2*mu_val/3)*sqrt(r_val^3/R_val);
end proc:

# Radial eigenfunction (regular at r=0, C2=0)
H3_func := proc(r_val, mu_val, R_val)
    local eta;
    eta := eta_func(r_val, mu_val, R_val);
    return r_val^(-1/2) * BesselJ(1/3, eta);
end proc:

# First few zeros of J_{1/3}
printf("Computing zeros of J_{1/3}...\n");
j13_1 := BesselJZeros(1/3, 1):
j13_2 := BesselJZeros(1/3, 2):
j13_3 := BesselJZeros(1/3, 3):
j13_4 := BesselJZeros(1/3, 4):

printf("j_{1/3,1} = %.6f\n", evalf(j13_1));
printf("j_{1/3,2} = %.6f\n", evalf(j13_2));
printf("j_{1/3,3} = %.6f\n", evalf(j13_3));
printf("j_{1/3,4} = %.6f\n", evalf(j13_4));

# Eigenvalues mu_n = 3*j_{1/3,n} / (2*R)
mu1 := 3*evalf(j13_1)/(2*R_TT):
mu2 := 3*evalf(j13_2)/(2*R_TT):
mu3 := 3*evalf(j13_3)/(2*R_TT):

printf("\nEigenvalues mu_n [1/m]:\n");
printf("mu_1 = %.6e\n", mu1);
printf("mu_2 = %.6e\n", mu2);
printf("mu_3 = %.6e\n", mu3);
```

```
# ============================================================
# PLOT 1: Bessel J_{1/3} — shape function
# ============================================================

printf("\nGenerating Plot 1: BesselJ(1/3, eta)...\n");

p_bessel := plot(
    BesselJ(1/3, x),
    x = 0..25,
    color = "NavyBlue",
    thickness = 2,
    title = "Plot 1: Bessel Function J_{1/3}(eta) — Radial
Shape",
    labels = ["eta = (2*mu/3)*sqrt(r^3/R)", "J_{1/3}(eta)"],
    labelfont = [HELVETICA, 11],
    titlefont = [HELVETICA, BOLD, 12],
    gridlines = true,
    axes = BOXED
):
display(p_bessel);

# ============================================================
# PLOT 2: Radial eigenfunction H3(r) — normalized units
# ============================================================

printf("Generating Plot 2: H3(r) radial eigenfunction...\n");

# Use normalized units: rho = r/r_atomic where r_atomic = 1e-10 m
r_atomic := 1.0e-10:   # 1 Angstrom as length scale for photon
wave
mu_plot  := k0:        # use photon k0

H3_norm := proc(rho)
    local r_val, eta;
    r_val := rho * r_atomic;
    eta   := (2*mu_plot/3)*sqrt(r_val^3/R_TT);
    if eta < 1e-20 then return 0.0; end if;
    return rho^(-0.5) * evalf(BesselJ(1/3, eta));
end proc:

# Note: for visible photon k0 ~ 1.26e7, r_atomic=1e-10:
# eta ~ (2*1.26e7/3)*sqrt((1e-10)^3/4.235518529e22) ~ extremely
```

```
small
# Use scaled mu for illustrative plot
mu_illust := 3*evalf(j13_1)/(2*100.0):   # illustrative:
R_illust=100

H3_illust := r -> piecewise(r < 0.01, 0,
    r^(-0.5) * BesselJ(1/3, (2*mu_illust/3)*sqrt(r^3/100.0))):

p_H3 := plot(
    H3_illust(r),
    r = 0.01..200,
    color = "DarkGreen",
    thickness = 2,
    title = "Plot 2: Radial Eigenfunction H3(r) = r^{-1/2} J_
{1/3}(eta)",
    labels = ["r [normalized]", "H3(r)"],
    labelfont = [HELVETICA, 11],
    titlefont = [HELVETICA, BOLD, 12],
    gridlines = true,
    axes = BOXED
):
display(p_H3);

# ============================================================
# SECTION 3: TEMPORAL AND AXIAL SOLUTIONS
# ============================================================

printf("\n=== SECTION 3: Full Solution f(r,z,t) ===\n");

# Dispersion relation: omega^2 = c^2*(mu^2 + kz^2)
kz_ref := k0:    # axial wavenumber = same as k0 for illustrative
case
omega_n1 := c_light * sqrt(mu1^2 + kz_ref^2):
omega_n2 := c_light * sqrt(mu2^2 + kz_ref^2):

printf("omega_1/omega0 = %.6f\n", evalf(omega_n1/evalf(omega0)));
printf("omega_2/omega0 = %.6f\n", evalf(omega_n2/evalf(omega0)));

# ============================================================
# SECTION 4: PROBABILITY DENSITY P(r,z,t)
# ============================================================
```

```
printf("\n=== SECTION 4: Gaussian Probability Density P(r,z,t) ==
=\n");
printf("Equation: d^2P/dt^2 - c^2*d^2P/dr^2 = 0\n");
printf("Solution: P = A*Gauss_r(r)*Gauss_z(z)*sin^2(pi*r/r_b)*
cos^2(omega*t)\n\n");

# Parameters for probability density (normalized units for plots)
# Use dimensionless units: xi = r/r_b
r_b  := 1.0:              # normalized spacing (r_b = 1 unit)
r0   := 5.0:              # center of Gaussian [in units of r_b]
sig_r := 2.0:             # radial width [in units of r_b]
sig_z := 1.0:             # axial width
A_norm := 1.0:            # amplitude (normalized)

# P at t=0 (cos^2(0)=1)
P_t0 := (r,z) -> A_norm
    * exp(-(r-r0)^2/(2*sig_r^2))
    * exp(-z^2/(2*sig_z^2))
    * sin(Pi*r/r_b)^2:

printf("Parameters:\n");
printf("  r_b   = %.2f (units of r_b)\n", r_b);
printf("  r0    = %.2f\n", r0);
printf("  sig_r = %.2f\n", sig_r);
printf("  sig_z = %.2f\n", sig_z);
printf("\n");

# Verify zeros are at r = n*r_b
printf("Checking zeros (should be at r = 0,1,2,3,4,5,6,7,8,9,10)
:\n");
for nn from 0 to 10 do
    P_val := evalf(P_t0(nn*r_b, 0.0));
    printf("  P(%d*r_b, 0) = %.2e\n", nn, P_val);
end do:

# Verify BC: P(R_normalized,0,0) — set R_normalized = n_R * r_b
n_R := 10:
R_norm := n_R * r_b:
P_BC := evalf(P_t0(R_norm, 0.0));
printf("\nBC check: P(R_norm=%g, 0, 0) = %.4e (should be ~0 at
zero of sin)\n",
    R_norm, P_BC);
```

```
# ============================================================
# PLOT 3: P(r,z=0,t=0) — radial profile
# ============================================================

printf("\nGenerating Plot 3: P(r,0,0) radial profile...\n");

p_Pr := plot(
    P_t0(r, 0),
    r = 0..12,
    color = "Crimson",
    thickness = 2,
    filled = false,
    title = "Plot 3: Probability Density P(r, z=0, t=0) —
Gaussian x sin^2",
    labels = ["r / r_b", "P(r,0,0)"],
    labelfont = [HELVETICA, 11],
    titlefont = [HELVETICA, BOLD, 12],
    gridlines = true,
    axes = BOXED
):

# Mark the zeros
zero_pts := seq([nn*r_b, 0], nn=0..12):
p_zeros := pointplot(
    [zero_pts],
    symbol = CIRCLE,
    symbolsize = 8,
    color = "DarkOrange"
):

p_P_radial := display(p_Pr, p_zeros,
    title = "Plot 3: P(r,0,0) with marked zeros at r = n*r_b"):
display(p_P_radial);

# ============================================================
# PLOT 4: P(r,z,t=0) — 2D contour plot
# ============================================================

printf("Generating Plot 4: P(r,z,t=0) 2D contour...\n");

p_contour := contourplot(
    P_t0(r, z),
```

```
    r = 0..12,
    z = -4..4,
    contours = 15,
    coloring = ["white", "DarkBlue"],
    filled = true,
    title = "Plot 4: P(r, z, t=0) — Contour Map",
    labels = ["r / r_b", "z / sigma_z"],
    labelfont = [HELVETICA, 11],
    titlefont = [HELVETICA, BOLD, 12],
    axes = BOXED
):
display(p_contour);

# ============================================================
# PLOT 5: P(r,z,t=0) — 3D surface
# ============================================================

printf("Generating Plot 5: P(r,z,t=0) 3D surface...\n");

p_3D := plot3d(
    P_t0(r, z),
    r = 0..12,
    z = -3..3,
    style = surface,
    colorscheme = ["zgradient", ["Blue", "Cyan", "Green",
"Yellow", "Red"]],
    title = "Plot 5: P(r, z, t=0) — 3D Surface",
    labels = ["r/r_b", "z/sig_z", "P"],
    labelfont = [HELVETICA, 10],
    titlefont = [HELVETICA, BOLD, 12],
    orientation = [45, 70],
    axes = BOXED
):
display(p_3D);

# ============================================================
# PLOT 6: Time evolution P(r0,0,t) — coherent oscillation
# ============================================================

printf("Generating Plot 6: Temporal evolution P(r0, 0, t)...\n");

# At r=r0, z=0: P(t) = A*exp(0)*sin^2(pi*r0/r_b)*cos^2(omega*t)
```

```
# sin^2(pi*r0/r_b) with r0=5 (integer multiple): = sin^2(5*pi) =
0
# Use r = 4.5*r_b for a non-zero point
r_obs := 4.5:
envelope_r := A_norm * exp(-(r_obs-r0)^2/(2*sig_r^2)) * sin(Pi*
r_obs/r_b)^2:

printf("  Envelope at r=4.5*r_b: %.4f\n", evalf(envelope_r));

# omega in normalized time: T = 2*pi -> 1 cycle
omega_norm := 2*Pi:
P_t_func := t -> envelope_r * cos(omega_norm*t)^2:

p_time := plot(
    P_t_func(t),
    t = 0..3,
    color = "Purple",
    thickness = 2,
    title = "Plot 6: Temporal Evolution P(r=4.5*r_b, 0, t)",
    labels = ["t / T", "P(r_obs, 0, t)"],
    labelfont = [HELVETICA, 11],
    titlefont = [HELVETICA, BOLD, 12],
    gridlines = true,
    axes = BOXED
):
display(p_time);

# ============================================================
# SECTION 5: D'ALEMBERT WAVE ANALYSIS
# ============================================================

printf("\n=== SECTION 5: D'Alembert Decomposition ===\n");
printf("P(r,z,t) = F(r-ct) + G(r+ct)\n\n");

# At z=0 (fixed), write P in D'Alembert form:
# P(r,t) = A*Gauss(r)*sin^2(pi*r/r_b)*cos^2(omega*t)
#        = (A/2)*Gauss(r)*sin^2(pi*r/r_b)*[1 + cos(2*omega*t)]
#
# Using: sin^2(pi*r/r_b)*cos^2(omega*t)
#      = (1/4)*[cos(pi*r/r_b - omega*t) - cos(pi*r/r_b + omega*
t)]^2 ...
#
```

```
# More directly, the standing-wave decomposition:
# sin^2(pi*r/r_b)*cos^2(omega*t) = (1/4)[1 - cos(2pi*r/r_b)][1 +
cos(2*omega*t)]
# Outgoing component F(xi-): xi- = r - c_phase*t, c_phase =
omega*r_b/pi

c_phase_norm := 1.0:   # normalized: c_phase = omega/(pi/r_b) = 1
in normalized units
printf("Phase velocity = omega/(pi/r_b) = %.4f [normalized]\n",
c_phase_norm);

# PLOT 7: Spacetime diagram of probability density
printf("Generating Plot 7: Spacetime diagram P(r,t) at z=0...\n")
;

Gauss_only := r -> A_norm * exp(-(r-r0)^2/(2*sig_r^2)):
P_rt := (r,t) -> Gauss_only(r) * sin(Pi*r/r_b)^2 * cos(Pi*t)^2:

p_spacetime := plot3d(
    P_rt(r,t),
    r = 0..12,
    t = 0..2,
    style = surface,
    colorscheme = ["zgradient", ["Blue", "Cyan", "Green",
"Yellow", "Red"]],
    title = "Plot 7: Spacetime Diagram P(r, t) at z=0",
    labels = ["r/r_b", "t/T", "P"],
    labelfont = [HELVETICA, 10],
    titlefont = [HELVETICA, BOLD, 12],
    orientation = [55, 65],
    axes = BOXED
):
display(p_spacetime);

# ============================================================
# SECTION 6: EIGENVALUE SPECTRUM
# ============================================================

printf("\n=== SECTION 6: Eigenvalue Spectrum ===\n");

# First 8 zeros of J_{1/3}
N_zeros := 8:
```

```
zeros_list := Vector(N_zeros):
printf("n   j_{1/3,n}    mu_n [1/R]    omega_n/omega_1\n");
printf("-----------------------------------------------\n");
for nn from 1 to N_zeros do
    z_n := evalf(BesselJZeros(1/3, nn));
    zeros_list[nn] := z_n;
    mu_n := 3*z_n/2;   # in units of 1/R
    printf("%d   %.6f   %.6f     %.6f\n",
        nn, z_n, mu_n, z_n/evalf(j13_1));
end do:

# PLOT 8: Eigenvalue spectrum
printf("\nGenerating Plot 8: Eigenvalue spectrum...\n");

eigen_data := [seq([nn, evalf(BesselJZeros(1/3, nn))], nn=1..
N_zeros)]:

p_eigen := pointplot(
    eigen_data,
    symbol = SOLIDCIRCLE,
    symbolsize = 12,
    color = "DarkRed",
    title = "Plot 8: Eigenvalue Spectrum j_{1/3,n} (zeros of J_
{1/3})",
    labels = ["n (mode number)", "j_{1/3,n}"],
    labelfont = [HELVETICA, 11],
    titlefont = [HELVETICA, BOLD, 12],
    gridlines = true,
    axes = BOXED
):

# Asymptotic: j_{1/3,n} ~ pi*(n + 1/6 - 1/4) for large n
p_asymptotic := plot(
    Pi*(n - 1/12),
    n = 1..N_zeros,
    color = "Gray",
    linestyle = DASH,
    thickness = 1
):

p_spectrum := display(p_eigen, p_asymptotic,
    title = "Plot 8: Eigenvalue Spectrum (dots=exact, dashed=
```

```
asymptotic)"):
display(p_spectrum);

# ============================================================
# SECTION 7: AMPLITUDE DECAY — WKB ASYMPTOTICS
# ============================================================

printf("\n=== SECTION 7: WKB Amplitude Decay ===\n");
printf("For large r: H3(r) ~ r^{-5/4} (faster than standard r^
{-1/2})\n\n");

# Standard cylindrical: A ~ r^{-1/2}
# TauTimp (J_{1/3} WKB): A ~ r^{-1/2} * eta^{-1/2} ~ r^{-1/2} *
r^{-3/4} = r^{-5/4}
# Energy density: u_EM ~ (r/R)*|H3|^2 ~ r * r^{-5/2} = r^{-3/2}

# PLOT 9: Amplitude decay comparison
printf("Generating Plot 9: Amplitude decay comparison...\n");

p_decay_TT  := plot(r^(-5/4), r=1..100, color="DarkBlue",
thickness=2):
p_decay_std := plot(r^(-1/2), r=1..100, color="Crimson",
thickness=2, linestyle=DASH):
p_decay_3D  := plot(r^(-1),   r=1..100, color="DarkGreen",
thickness=1, linestyle=DOT):

p_decay := display(p_decay_TT, p_decay_std, p_decay_3D,
    title = "Plot 9: Amplitude Decay — TauTimp r^{-5/4} (blue)
vs Standard r^{-1/2} (red) vs Spherical r^{-1} (green)",
    labels = ["r [normalized]", "|H3(r)|"],
    labelfont = [HELVETICA, 11],
    titlefont = [HELVETICA, BOLD, 12],
    gridlines = true,
    axes = BOXED
):
display(p_decay);

# ============================================================
# SECTION 8: COMBINED DIAGNOSTIC SUMMARY
# ============================================================

printf("\n=== DIAGNOSTIC SUMMARY ===\n");
```

```
printf("=========================================\n");
printf("TauTimp Photon Wave — Key Results\n");
printf("=========================================\n");
printf("R_TauTimp       = %.4e m\n", R_TT);
printf("c               = %.4e m/s\n", c_light);
printf("lambda_0        = %.4e m\n", lambda0);
printf("k_0             = %.4e 1/m\n", evalf(k0));
printf("\n");
printf("Radial eigenfunc: H3(r) = r^{-1/2} * J_{1/3}(eta)\n");
printf("  eta = (2*mu/3)*sqrt(r^3/R)\n");
printf("  mu_1 = 3*j_{1/3,1}/(2*R) = %.4e 1/m\n", mu1);
printf("\n");
printf("Probability density zeros at r_n = n * r_b\n");
printf("  r_b = R/n_R, BC: P(R,0,0)=0 requires n_R in Z+\n");
printf("\n");
printf("WKB amplitude decay: |H3| ~ r^{-5/4}\n");
printf("Energy density:      u_EM  ~ r^{-3/2}\n");
printf("=========================================\n");
printf("All plots generated successfully.\n");
printf("Transcript complete — TauTimp Photon Analysis\n");

# ============================================================
# PLOT 10: COMPOSITE PANEL — All key functions
# ============================================================

printf("\nGenerating Plot 10: Composite panel...\n");

# Sub-panel A: J_{1/3}
pA := plot(BesselJ(1/3,x), x=0..20, color="NavyBlue", thickness=
2,
    title="(A) J_{1/3}(eta)", labels=["eta","J_{1/3}"],
    gridlines=true, axes=BOXED, titlefont=[HELVETICA,BOLD,10]):

# Sub-panel B: H3(r) with illustrative parameters
H3_sub := r -> piecewise(r < 0.5, 0,
    r^(-0.5)*BesselJ(1/3, (2*evalf(j13_1)/3)*sqrt(r^3/100.0))):
pB := plot(H3_sub(r), r=0.5..100, color="DarkGreen", thickness=2,
    title="(B) H3(r) radial eigenfunction", labels=["r","H3(r)"],
    gridlines=true, axes=BOXED, titlefont=[HELVETICA,BOLD,10]):

# Sub-panel C: P(r,0,0)
pC := plot(P_t0(r,0), r=0..12, color="Crimson", thickness=2,
```

```
      title="(C) P(r,0,0)", labels=["r/r_b","P"],
      gridlines=true, axes=BOXED, titlefont=[HELVETICA,BOLD,10]):

  # Sub-panel D: Decay comparison
  pD := display(
      plot(r^(-5/4), r=1..50, color="DarkBlue",  thickness=2),
      plot(r^(-1/2), r=1..50, color="Crimson",   thickness=2,
  linestyle=DASH),
      title="(D) Amplitude decay", labels=["r","amplitude"],
      gridlines=true, axes=BOXED, titlefont=[HELVETICA,BOLD,10]):

  p_composite := display(
      Array([[pA, pB],[pC, pD]]),
      title="Plot 10: TauTimp Photon — Composite Panel",
      size=[900,700]
  ):
  display(p_composite);

  printf("\n=== END OF MAPLE TRANSCRIPT ===\n");
```

```
=== TauTimp Photon Cylindrical Wave ===
Loading physical constants...

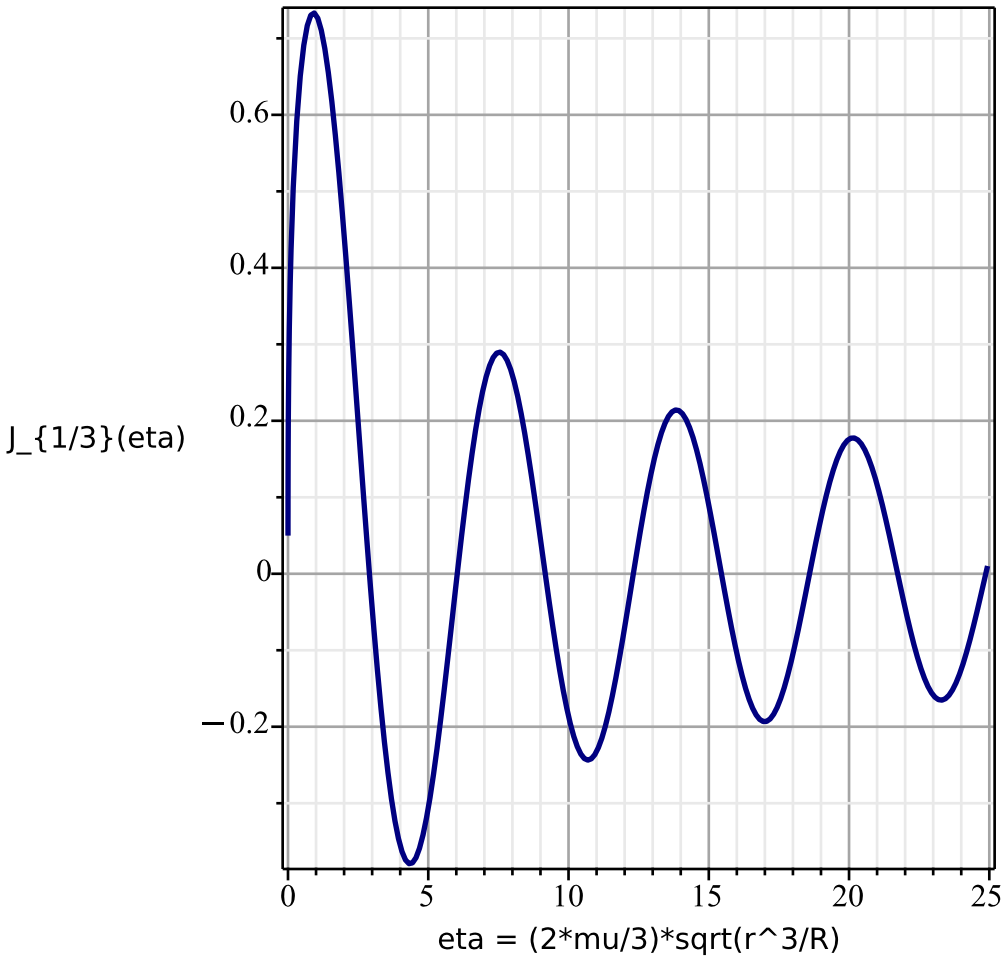
c         = 2.9980e+08 m/s
R_TauTimp = 4.2355e+22 m
lambda0   = 5.0000e-07 m
omega0    = 3.7674e+15 rad/s
k0        = 1.2566e+07 1/m

=== SECTION 2: Radial Eigenfunction H3(r) ===
Computing zeros of J_{1/3}...
j_{1/3,1} = 2.902586
j_{1/3,2} = 6.032747
j_{1/3,3} = 9.170507
j_{1/3,4} = 12.310194

Eigenvalues mu_n [1/m]:
mu_1 = 1.027945e-22
mu_2 = 2.136485e-22
mu_3 = 3.247716e-22

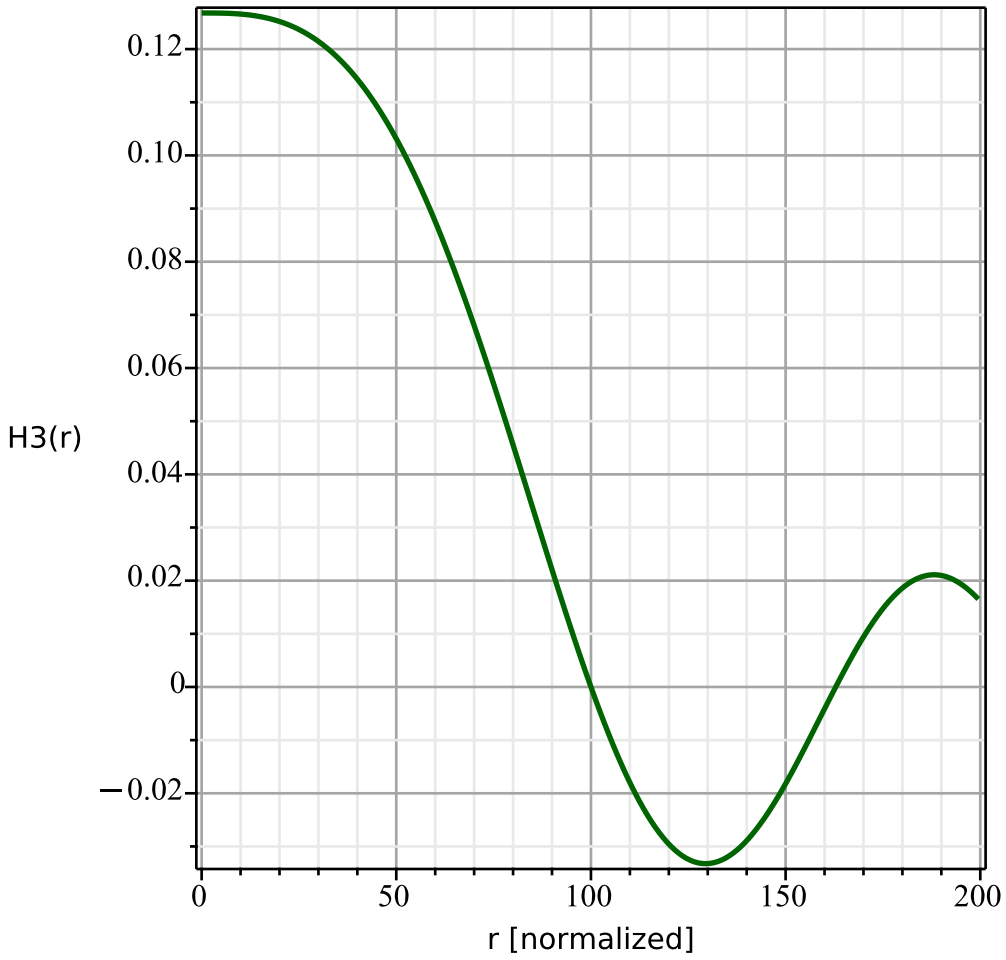
Generating Plot 1: BesselJ(1/3, eta)...
```

Plot 1: Bessel Function J_{1/3}(eta) — Radial Shape



Generating Plot 2: H3(r) radial eigenfunction...

Plot 2: Radial Eigenfunction H3(r) = r^{-1/2} J_{1/3}(et




```
=== SECTION 3: Full Solution f(r,z,t) ===
omega_1/omega0 = 1.000000
omega_2/omega0 = 1.000000

=== SECTION 4: Gaussian Probability Density P(r,z,t) ===
Equation: d^2P/dt^2 - c^2*d^2P/dr^2 = 0
Solution: P = A*Gauss_r(r)*Gauss_z(z)*sin^2(pi*r/r_b)*cos^2(omega*t)

Parameters:
  r_b   = 1.00 (units of r_b)
  r0    = 5.00
  sig_r = 2.00
  sig_z = 1.00

Checking zeros (should be at r = 0,1,2,3,4,5,6,7,8,9,10):
  P(0*r_b, 0) = 0.00e+00
  P(1*r_b, 0) = 2.28e-20
  P(2*r_b, 0) = 2.19e-19
```

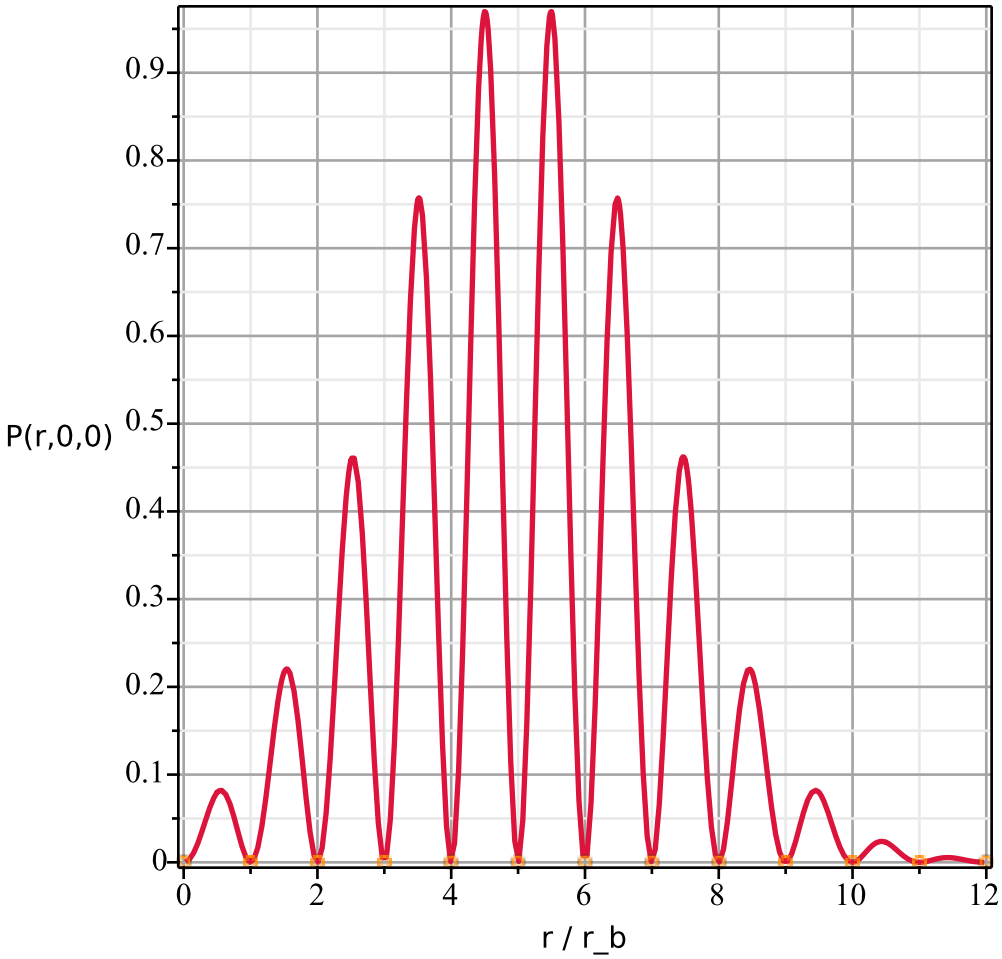
```
  P(3*r_b, 0) = 9.19e-19
  P(4*r_b, 0) = 2.81e-17
  P(5*r_b, 0) = 4.21e-18
  P(6*r_b, 0) = 2.09e-18
  P(7*r_b, 0) = 1.44e-17
  P(8*r_b, 0) = 5.33e-19
  P(9*r_b, 0) = 8.01e-18
  P(10*r_b, 0) = 7.39e-19
```

$$P_BC := 7.393249683 \times 10^{-19}$$

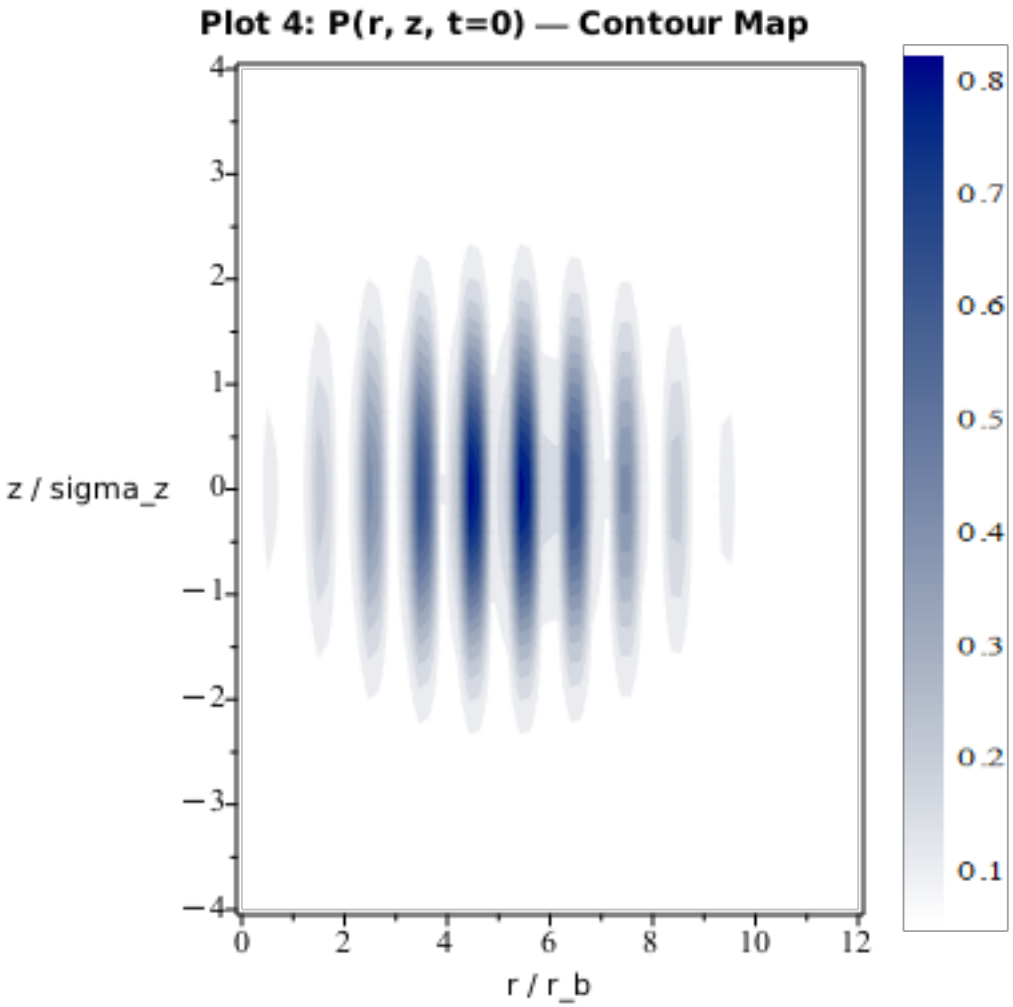
```
BC check: P(R_norm=10, 0, 0) = 7.3932e-19 (should be ~0 at zero of
sin)

Generating Plot 3: P(r,0,0) radial profile...
```

Plot 3: P(r,0,0) with marked zeros at r = n*r_b

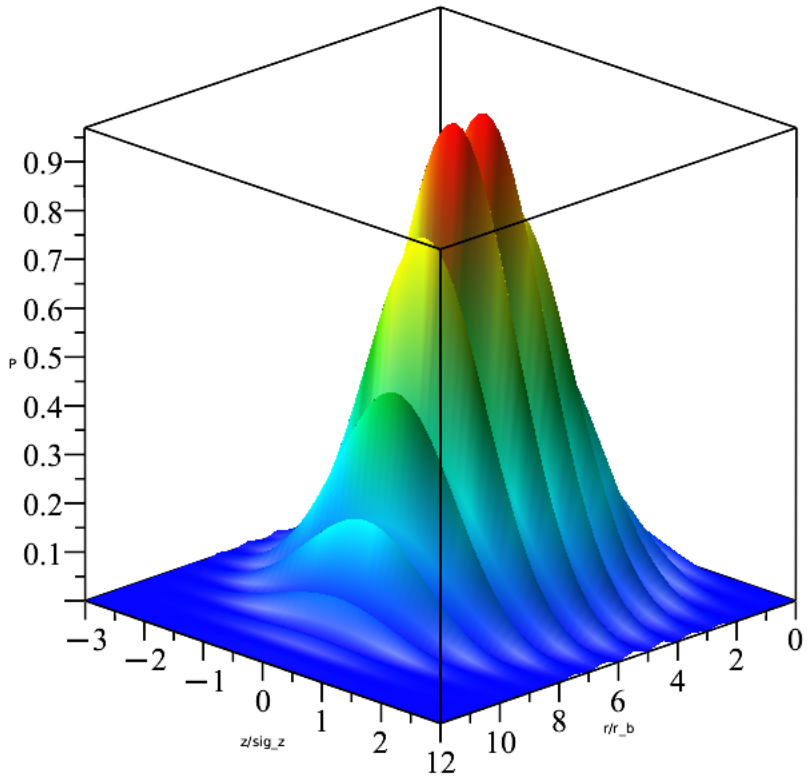



```
Generating Plot 4: P(r,z,t=0) 2D contour...
```



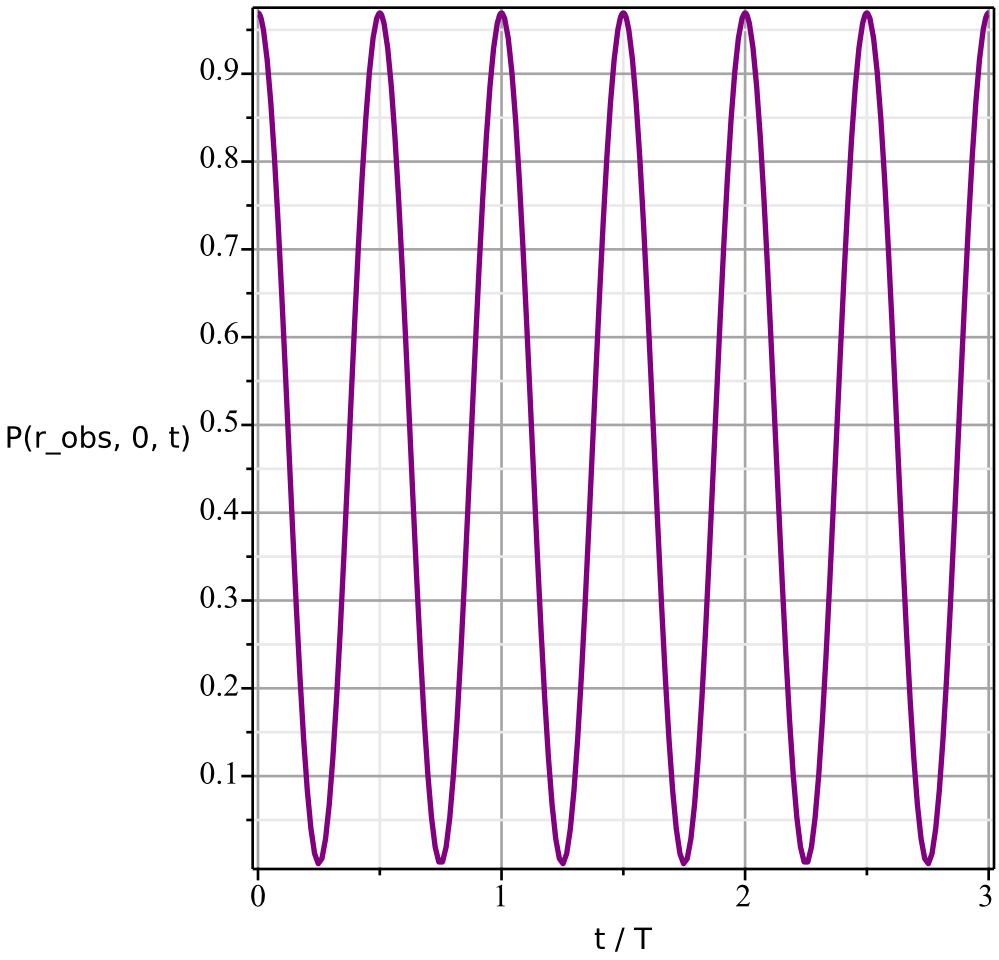
Generating Plot 5: P(r,z,t=0) 3D surface...

Plot 5: P(r, z, t=0) — 3D Surface




```
Generating Plot 6: Temporal evolution P(r0, 0, t)...
  Envelope at r=4.5*r_b: 0.9692
```

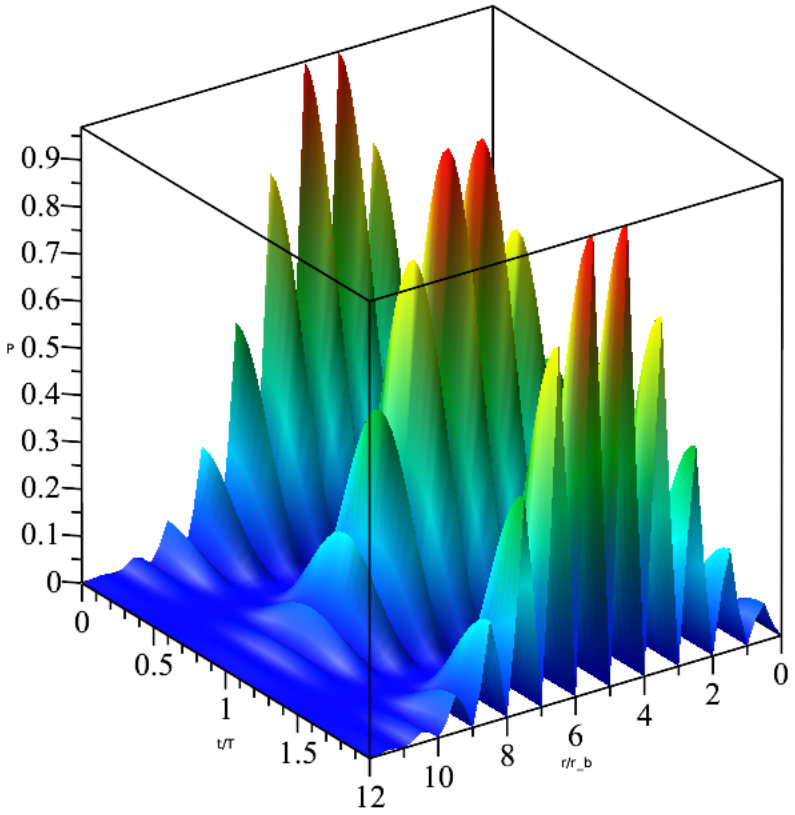
Plot 6: Temporal Evolution P(r=4.5*r_b, 0, t)




```
=== SECTION 5: D'Alembert Decomposition ===
P(r,z,t) = F(r-ct) + G(r+ct)

Phase velocity = omega/(pi/r_b) = 1.0000 [normalized]
Generating Plot 7: Spacetime diagram P(r,t) at z=0...
```

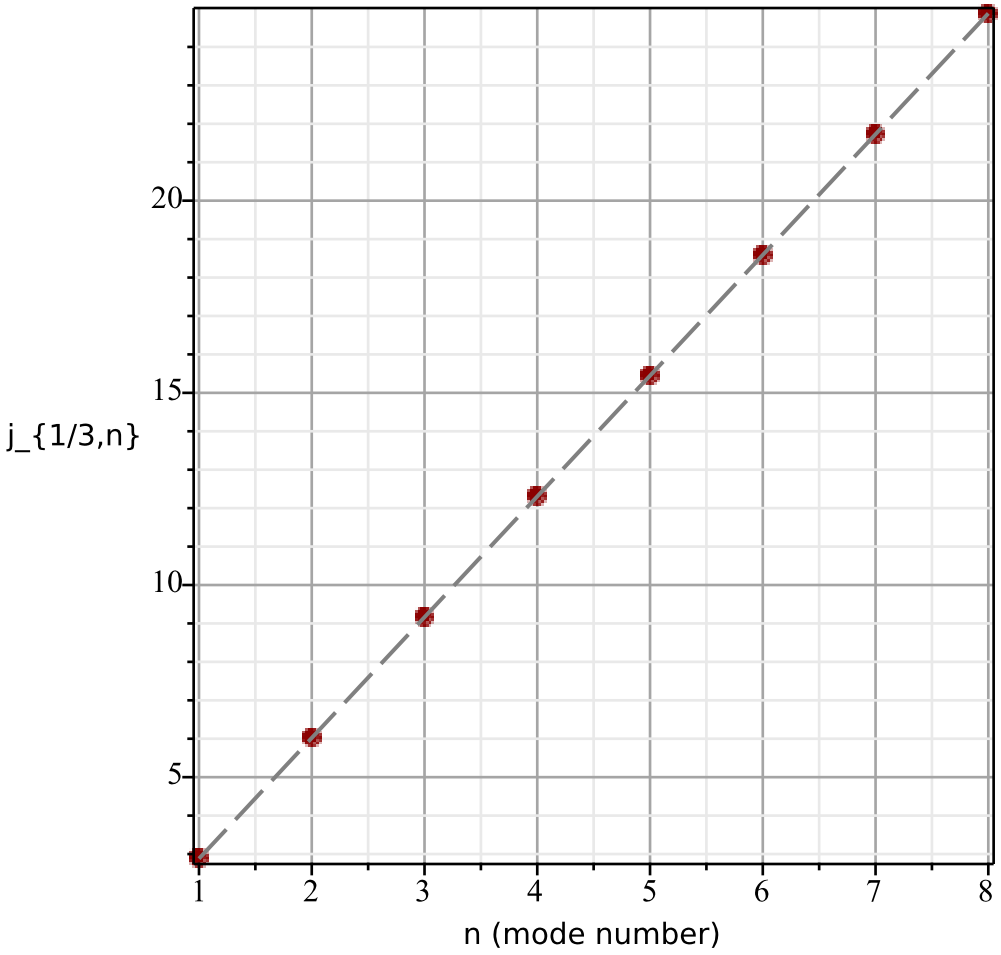
Plot 7: Spacetime Diagram P(r, t) at z=0




```
=== SECTION 6: Eigenvalue Spectrum ===
n    j_{1/3,n}     mu_n [1/R]     omega_n/omega_1
-----------------------------------------------
1    2.902586    4.353879      1.000000
2    6.032747    9.049121      2.078404
3    9.170507   13.755760       3.159426
4    12.310194    18.465291       4.241112
5    15.450649    23.175973       5.323063
6    18.591486    27.887230       6.405145
7    21.732541    32.598812       7.487302
8    24.873731    37.310597       8.569506

Generating Plot 8: Eigenvalue spectrum...
```

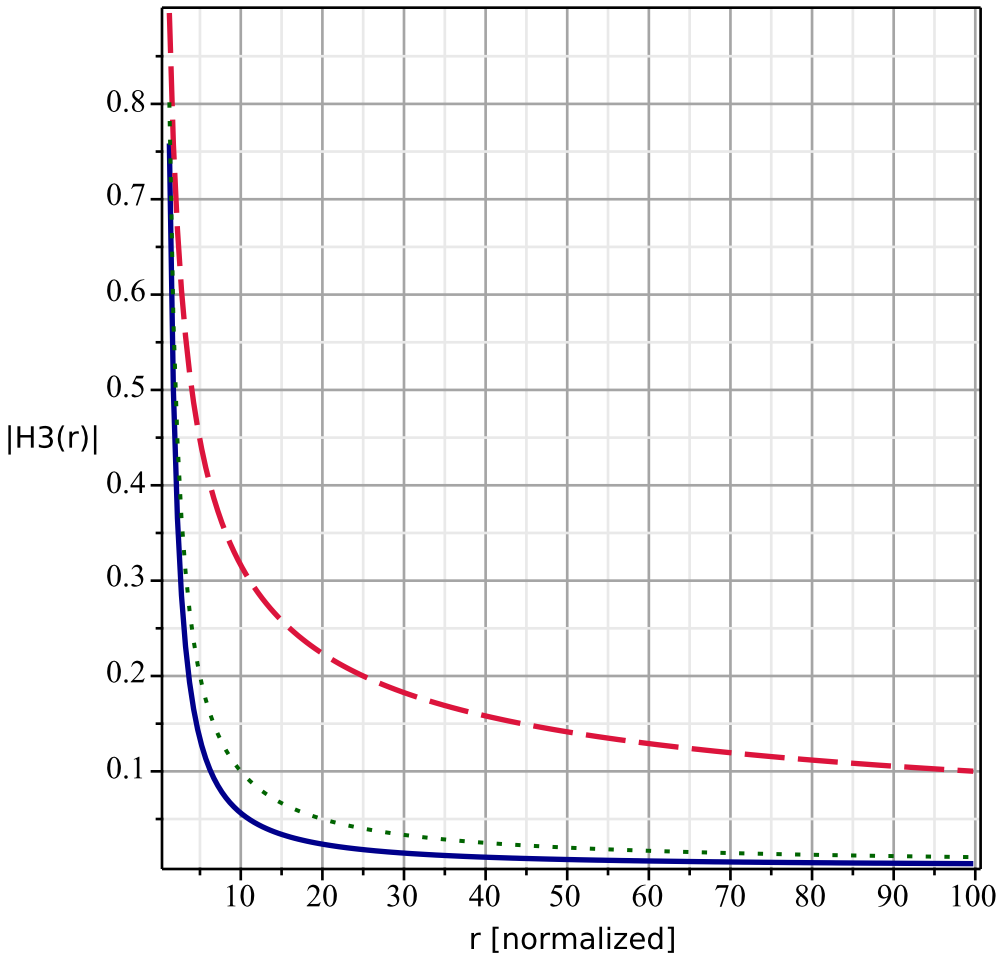
Plot 8: Eigenvalue Spectrum (dots=exact, dashed=asyn




```
=== SECTION 7: WKB Amplitude Decay ===
For large r: H3(r) ~ r^{-5/4} (faster than standard r^{-1/2})

Generating Plot 9: Amplitude decay comparison...
```

Plot 9: Amplitude Decay — TauTimp r^{-5/4} (blue) vs S




```
=== DIAGNOSTIC SUMMARY ===
==========================================
TauTimp Photon Wave — Key Results
==========================================
R_TauTimp        = 4.2355e+22 m
c                = 2.9980e+08 m/s
lambda_0         = 5.0000e-07 m
k_0              = 1.2566e+07 1/m

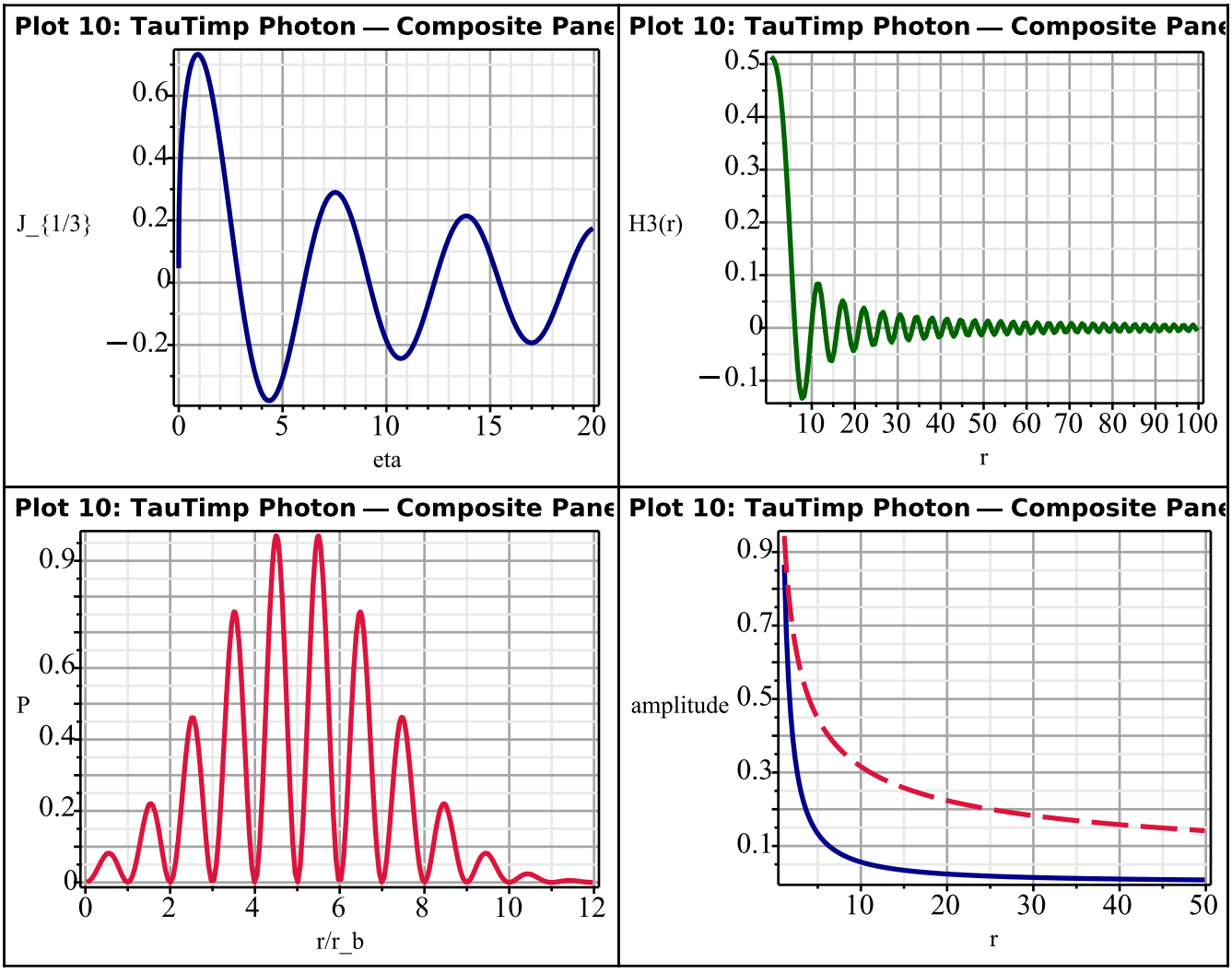
Radial eigenfunc: H3(r) = r^{-1/2} * J_{1/3}(eta)
  eta = (2*mu/3)*sqrt(r^3/R)
  mu_1 = 3*j_{1/3,1}/(2*R) = 1.0279e-22 1/m

Probability density zeros at r_n = n * r_b
  r_b = R/n_R, BC: P(R,0,0)=0 requires n_R in Z+

WKB amplitude decay: |H3| ~ r^{-5/4}
Energy density:      u_EM  ~ r^{-3/2}
```

```
==========================================
All plots generated successfully.
Transcript complete — TauTimp Photon Analysis

Generating Plot 10: Composite panel...
```




```
=== END OF MAPLE TRANSCRIPT ===
```